

Lab 3: Understanding Pod Lifecycle and Pod States in Kubernetes

Objective

The objective of this lab is to understand:

- What is Pod Lifecycle?
 - Different Pod States
 - How Pods move from creation to termination
 - Common Pod Errors
 - Troubleshooting commands
 - Interview-focused concepts
-

What is Pod Lifecycle?

A Pod does not directly enter the Running state after creation.

When a Pod is created, Kubernetes moves it through multiple phases.

The complete Pod lifecycle describes the journey of a Pod from creation to termination.

Pod Lifecycle Flow

```
Pending
  ↓
ContainerCreating
  ↓
Running
  ↓
Succeeded / Failed
```

In case of problems:

```
ErrImagePull
ImagePullBackOff
CrashLoopBackOff
Unknown
```

Pod State 1: Pending

Description

The Pod has been accepted by Kubernetes but has not yet started running.

Kubernetes is performing tasks such as:

- Finding a suitable Node
- Scheduling the Pod
- Allocating resources

Example

NAME	READY	STATUS
nginx-pod	0/1	Pending

Pod State 2: ContainerCreating

Description

The Pod has been scheduled to a Node.

Kubernetes is now:

- Pulling the image
- Creating networking
- Preparing storage
- Starting the container

Example

NAME	READY	STATUS
nginx-pod	0/1	ContainerCreating

Pod State 3: Running

Description

The container has started successfully.

The Pod is operational and serving requests.

Example

NAME	READY	STATUS
nginx-pod	1/1	Running

Important Note

Running does not always mean the application is healthy.

Application health is checked using probes such as:

- Liveness Probe
 - Readiness Probe
 - Startup Probe
-

Pod State 4: Succeeded

Description

The Pod completed its task successfully and exited with exit code 0.

This is common for:

- Batch Jobs
- One-time scripts
- Data processing tasks

Example

STATUS = Succeeded

Pod State 5: Failed

Description

The container terminated with a non-zero exit code.

Possible reasons:

- Application error
- Invalid command
- Script failure

Example

STATUS = Failed

Pod State 6: Unknown

Description

Kubernetes cannot determine the current status of the Pod.

Usually happens when communication between the Control Plane and Node is lost.

Example

STATUS = Unknown

Common Pod Error States

These states are frequently asked in interviews.

ErrImagePull

Description

Kubernetes attempted to pull the container image but failed.

Possible reasons:

- Wrong image name
- Private image without credentials
- Registry connectivity issue

Example

image: nginx12345

Output:

ErrImagePull

ImagePullBackOff

Description

Kubernetes repeatedly tries to pull the image but continues to fail.

Usually occurs after ErrImagePull.

Flow:

```
ErrImagePull  
  ↓  
ImagePullBackOff
```

CrashLoopBackOff

Description

The container starts successfully but crashes repeatedly.

Kubernetes keeps restarting it.

Flow:

```
Start  
  ↓  
Crash  
  ↓  
Restart  
  ↓  
Crash  
  ↓  
Restart
```

Common Reasons

- Application startup failure
 - Invalid configuration
 - Missing environment variables
 - Dependency issues
-

Interview Questions

Difference Between Pending and ContainerCreating

Pending

- Pod is not yet assigned to a Node
- Scheduling is still in progress

ContainerCreating

- Node has already been assigned
 - Container setup process has started
-

Troubleshooting Commands

Check Pod Status

```
kubectl get pods
```

Detailed Pod Information

```
kubectl describe pod nginx-pod
```

View Pod Logs

```
kubectl logs nginx-pod
```

Access Pod Terminal

```
kubectl exec -it nginx-pod -- /bin/sh
```

Pod Lifecycle Summary Table

State	Meaning
Pending	Waiting for scheduling/resources
ContainerCreating	Container setup in progress
Running	Container running successfully

State	Meaning
Succeeded	Task completed successfully
Failed	Task failed
Unknown	Status cannot be determined
ErrImagePull	Image pull failed
ImagePullBackOff	Repeated image pull failure
CrashLoopBackOff	Continuous container crash and restart

Key Learnings

By completing this lab, we learned:

- Pod Lifecycle
- Pod States
- Pod Scheduling Process
- Container Creation Process
- Common Failure States
- Troubleshooting Commands
- Interview-Oriented Concepts

Practical Exercise

Step 1: Create a Pod

Create a file named:

pod-lifecycle.yaml

```
apiVersion: v1
kind: Pod

metadata:
  name: nginx-pod

spec:
```

```
containers:
- name: nginx-container
  image: nginx
```

Step 2: Create the Pod

```
kubectl apply -f pod-lifecycle.yaml
```

Expected Output:

```
pod/nginx-pod created
```

Step 3: Check Pod Status

```
kubectl get pods
```

Possible States:

```
Pending
ContainerCreating
Running
```

Observe how the Pod transitions between states.

Step 4: Watch Pod Lifecycle in Real Time

```
kubectl get pods -w
```

Example:

NAME	READY	STATUS
nginx-pod	0/1	Pending
nginx-pod	0/1	ContainerCreating
nginx-pod	1/1	Running

Step 5: Inspect Pod Details

```
kubectl describe pod nginx-pod
```

Verify:

- Events
- Image
- Node Name
- Pod IP
- Container Status

Step 6: Check Logs

```
kubectl logs nginx-pod
```

If the application writes logs, they will appear here.

Step 7: Access the Container

```
kubectl exec -it nginx-pod -- /bin/sh
```

Verify the container is running correctly.

Exit:

```
exit
```

Step 8: Delete the Pod

```
kubectl delete pod nginx-pod
```

Verify deletion:

```
kubectl get pods
```

Expected:

```
No resources found
```